

MLOps 기말 프로젝트 최종 보고서

다이캐스팅 정상/불량 예측을 위한 MLOps 기반 AI 서비스

End-to-End MLOps Pipeline for Die-Casting Defect Prediction

아주대학교 · AI융합실전프로젝트3 · 8조

팀원: 김병근 (Data) · Zhang Xin (Modeling/XAI) · 심재광 (MLOps/Serving/발표)

제출일: 2026-06-14

https://github.com/jksim1833/2026_AJOU_mlops_team8

목차

목차.....	2
요약 (Executive Summary).....	3
1. 프로젝트 개요.....	4
1.1 문제 정의.....	4
1.2 대상 사용자 · 입출력	4
1.3 MVP 범위	4
2. 데이터 이해와 전처리	5
2.1 데이터 소스와 라벨 생성	5
2.2 중복 제거 · 클래스 분포	5
2.3 데이터 분할 (Train/Valid/Test)	5
2.4 EDA 및 Feature 분석	6
3. MLOps 구조	8
3.1 Source Code Version Control (Git/GitHub)	8
3.2 Data Version Control (DVC).....	9
3.3 Experiment Tracking & Model Governance (MLflow)	10
3.4 컨테이너화 (Docker).....	11
3.5 End-to-End 재현성 보장	11
4. 모델 실험 및 성능 비교	13
4.1 AutoML-lite Leaderboard	13
4.2 Logistic Regression 튜닝	13
4.3 결정 임계값(Threshold) 튜닝	14
5. XAI 및 오류 분석.....	16
5.1 전역 해석 (Global Explanation)	16
5.2 지역 해석 (Local Explanation)	17
5.3 모델 한계와 개선 방향	18
6. Serving 및 Demo.....	19
6.1 FastAPI Endpoint.....	19
6.2 Streamlit Web UI	19
6.3 Docker 배포 및 검증	20
7. 결론 및 Future Work.....	22
7.1 결론	22
7.2 Future Work.....	22

요약 (Executive Summary)

본 프로젝트는 한국지능정보사회진흥원 KAMP 다이캐스팅(die-casting) 공정·센서 데이터를 사용해 제품의 정상/불량(normal/defect)을 예측하는 이진 분류 AI 서비스를, 데이터 버전관리부터 실험 추적·모델 서빙·컨테이너 배포까지 MLOps 전 과정으로 구현한 결과를 정리한다. 단순 모델 성능 비교가 아니라 '하나의 AI 서비스를 개발·운영하는 관점'에서 재현 가능한 파이프라인을 구축하는 데 초점을 두었다.

항목	핵심 결과
문제 정의	공정 센서값으로 제품 불량 여부를 예측하고 근거(XAI)를 제시
데이터	KAMP Product 1, 중복 제거 후 2,515건 (normal 1,960 / defect 555)
Champion 모델	StandardScaler + Logistic Regression (logistic_champion_v1)
성능 (Test)	F1 0.488 · Recall 0.759 · ROC-AUC 0.758 (threshold 0.49)
MLOps 스택	Git/GitHub · DVC · MLflow · FastAPI · Streamlit · Docker
재현성	DVC 4-stage pipeline, clean clone에서 dvc pull 복원 검증

1. 프로젝트 개요

1.1 문제 정의

한 문장 정의: 다이캐스팅 공정/센서 데이터로부터 제품의 정상/불량을 예측하고, 그 판단 근거를 XAI로 함께 제시하는 AI 서비스.

다이캐스팅은 고온 용탕을 고압으로 금형에 주입하는 공정으로, 온도·압력·분사 시간 등 다수의 공정 변수가 제품 품질에 영향을 준다. 현장의 공정 엔지니어는 이러한 센서값을 보고 불량 가능성을 판단해야 하지만, 변수 간 상호작용이 복잡해 사람의 직관만으로는 일관된 판단이 어렵다. 본 서비스는 공정값을 입력하면 정상/불량 예측과 주요 기여 변수를 반환하여 의사결정을 보조한다.

1.2 대상 사용자 · 입출력

구분	내용
대상 사용자	다이캐스팅 공정 엔지니어 / 품질 관리자
입력 (Input)	28개 공정·센서 feature (속도, 압력, 분사 시간, 온도, 습도 등)
출력 (Output)	정상/불량 예측, 클래스별 확률, 상위 기여 feature, 모델·데이터 버전
성공 기준	validation F1/ROC-AUC, MLflow 실험 기록, FastAPI+UI demo, Docker 실행 구조

1.3 MVP 범위

불량 종류를 세분하지 않고 정상/불량 이진 분류로 단순화하여, 제한된 기간 안에 end-to-end MLOps 흐름을 완성하는 것을 MVP 목표로 삼았다. 멀티클래스 결함 분류, 실시간 스트리밍 추론, 클라우드 배포는 future work로 분리했다.

2. 데이터 이해와 전처리

2.1 데이터 소스와 라벨 생성

원본 데이터는 KAMP 다이캐스팅 Product 1 데이터셋(DieCasting_Quality_Raw_Data_product1.csv)이다. 원본에는 공정/센서 측정값과 함께 결함 유형별 판정 컬럼(Short_Shot, Bubble, Blow_Hole, Crack 등)이 포함되어 있다.

이진 라벨 규칙: 결함 컬럼들의 합이 0보다 크면 defect(1), 모두 0이면 normal(0)으로 defect_label을 생성했다. 라벨 생성 직후 26개의 결함 판정 컬럼을 feature에서 제거하여 label leakage(정답 누출)를 차단했다.

제거한 결함 컬럼(26개): Short_Shot, Bubble, Exfoliation, Blow_Hole, Stain, Dent, Deformation, Contamination, Impurity, Crack, Scratch, Buring_Mark, Inclusions의 _1 / _2 버전.

2.2 중복 제거 · 클래스 분포

항목	값	비고
원본 행 수	4,207	raw
완전 중복 제거	-1,692	동일 processed row
최종 행 수	2,515	dedup 후
feature/컬럼 수	28 / 29	target 포함 29
normal (0)	1,960	77.93%
defect (1)	555	22.07%
결측치	0	median imputation 결과 0
상수 feature	8	Min/Max 류 (분산 0)

결측치는 numeric 변환 후 feature별 중앙값으로 대체했으나 실제 대체 건수는 0이었다.

Air/Coolant/Factory의 _Min/_Max 8개 컬럼은 분산이 0인 상수 feature로 확인되었다(모델 입력에는 유지하되 해석 시 제외).

2.3 데이터 분할 (Train/Valid/Test)

defect_label 기준 stratified split을 적용해 클래스 비율을 모든 분할에서 유지했다(test 15%,

validation 15% of remaining, random_state=42). 세 분할 간 완전 동일 행 중복(exact overlap)은 0건으로 검증되어 data leakage가 없음을 확인했다.

분할	전체	normal	defect
Train	1,759	1,370	389
Validation	378	295	83
Test	378	295	83

2.4 EDA 및 Feature 분석

표준화 평균차($SMD = |\mu_{\text{defect}} - \mu_{\text{normal}}| / \sigma_{\text{pop}}$) 기준으로 클래스 분리력이 큰 상위 8개 feature를 선정해 분포·상관을 분석했다.

Feature	SMD	target 상관
Factory_Humidity	0.715	-0.297
Factory_Temp	0.581	+0.241
Spray_Time	0.449	-0.186
Casting_Pressure	0.408	-0.169
Cylinder_Pressure	0.407	-0.169
Pressure_Rise_Time	0.302	+0.125
Spray_2_Time	0.275	+0.114
Air_Pressure	0.254	+0.105

Factory_Humidity가 가장 강한 분리력을 보이며 불량과 음의 상관(습도가 낮을수록 불량 경향)을 나타냈다. Factory_Humidity와 Factory_Temp는 -0.95로 강한 음의 상관, Casting_Pressure와 Cylinder_Pressure는 +0.999로 거의 동일한 정보를 담아 다중공선성이 존재함을 확인했다.

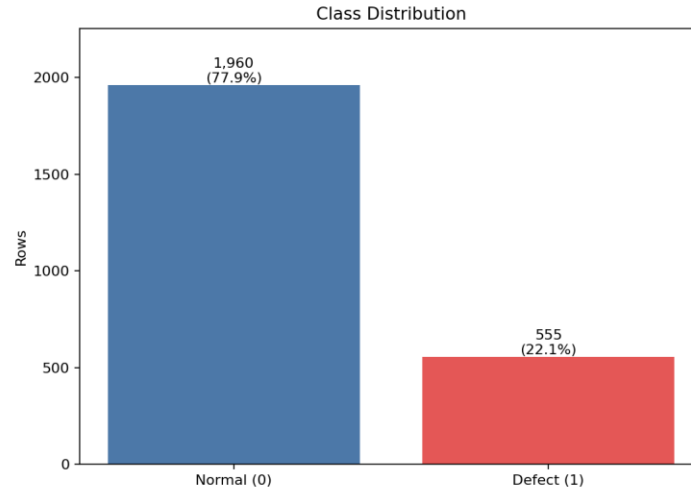


그림 2-1. 클래스 분포 (normal 77.9% / defect 22.1%) — class imbalance 존재

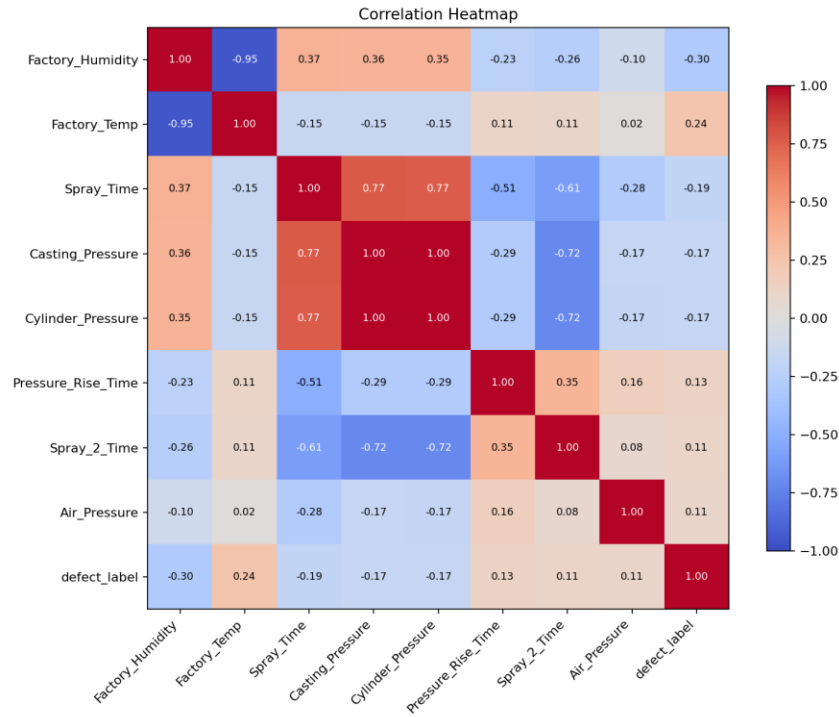


그림 2-2. 선정 feature 간 상관 히트맵 — 압력 변수 간 강한 상관(다중공선성)

3. MLOps 구조

본 프로젝트의 MLOps 설계 원칙은 "같은 입력에서 항상 같은 결과가 나오고, 그 과정과 산출물이 모두 추적되며, 모델을 코드·환경과 함께 통째로 재현·배포할 수 있어야 한다"는 것이다. 이를 위해 (1) 코드 버전관리(Git), (2) 데이터·파이프라인 버전관리(DVC), (3) 실험 추적·모델 거버넌스(MLflow), (4) 환경 고정 서빙(Docker)의 네 계층을 결합했다. 네 도구는 독립적으로 동작하지 않고, 아래와 같이 하나의 재현 가능한 흐름으로 맞물린다.

전체 데이터 흐름: raw CSV → (DVC) prepare_data → train/valid/test →

train_binary-compare_baselines_xai → tune_logistic → champion model.joblib → (MLflow) 실험 기록-champion 태깅 → (Docker) image build → FastAPI/Streamlit 서빙. 각 화살표의 입력·출력 해시와 파라미터가 버전관리되어, 임의의 시점 결과를 동일하게 복원할 수 있다.

계층	구성	핵심 역할
Source Control	Git / GitHub (sim → main)	코드·설정 이력, 협업
Data Versioning	DVC 4-stage pipeline + local remote	데이터·산출물 추적, 재실행
Experiment Tracking	MLflow (sqlite backend + artifact store)	실험 비교, champion 거버넌스
Serving	FastAPI + Streamlit + Docker	환경 고정, 추론 제공

3.1 Source Code Version Control (Git/GitHub)

원격 저장소 **2026_AJOU_mlops_team8**에서 통합 브랜치 sim에 작업을 누적하고 최종 제출 시 default branch인 main에 반영하는 단순 브랜치 전략을 사용한다. 팀 작업은 의미 단위 커밋(데이터 준비, baseline, 튜닝, Docker 검증 등)으로 분리해 변경 이력을 추적 가능하게 유지했다.

버전관리 경계: .gitignore로 data/raw, data/processed, artifacts/, mlflow.db 등 대용량 데이터와 생성물은 Git에서 제외하고, 코드(src/)·설정(configs/params.yaml, dvc.yaml)·DVC 메타파일(*.dvc, dvc.lock)만 Git으로 관리한다. 이렇게 "코드는 Git, 데이터/모델은 DVC"로 책임을 분리하면 저장소가 가벼워지고, 동시에 어떤 코드 커밋이 어떤 데이터·모델 버전과 짝을 이루는지 dvc.lock의 해시를 통해 1:1로 추적된다.

단일 진실 원천(Single Source of Truth): split 비율, random_state(42), 튜닝 search 횟수·CV

fold-threshold 탐색 범위, MLflow experiment 이름 등 모든 실행 파라미터를 configs/params.yaml 한 곳에 모았다. 코드에 숫자를 흘뿌리지 않고 설정으로 외부화함으로써, 파라미터 변경이 곧 DVC 의존성 변경으로 감지되어 재현성과 실험 관리가 동시에 보장된다.

3.2 Data Version Control (DVC)

동작 원리: DVC는 데이터·모델 파일 자체를 Git에 넣는 대신, 파일 내용을 해시(MD5)로 식별하는 content-addressed 캐시(.dvc/cache)에 저장하고, Git에는 그 해시를 가리키는 작은 메타파일만 커밋한다. 따라서 수백 MB의 데이터가 바뀌어도 Git diff는 해시 한 줄만 바뀌며, 특정 커밋으로 checkout하면 그 시점의 정확한 데이터·모델이 복원된다.

파이프라인(DAG)과 증분 실행: dvc.yaml에 4개 stage를 의존성(deps)과 산출물(outs)로 선언하면, DVC는 이를 방향성 비순환 그래프(DAG)로 해석한다. dvc repro 실행 시 각 stage의 입력 코드·데이터·params 해시를 dvc.lock에 기록된 값과 비교해, 변경이 감지된 stage와 그 하위만 다시 실행한다 (증분 실행). 즉 전처리만 바뀌면 전체가 아니라 영향받는 단계만 재계산되어 빠르고 일관된 재현이 가능하다.

Stage	역할	주요 산출물(outs)
prepare_data	라벨 생성·중복 제거·stratified 분할 ·EDA	processed CSV, train/valid/test, data_profile.json, EDA plots
train_binary	RandomForest baseline 학습 + MLflow 로깅	rf_baseline.joblib, 지표/플롯
compare_baselines_xai	5종 후보 leaderboard + SHAP 해석	baseline_comparison.json, SHAP 산 출물
tune_logistic	5-fold CV 튜닝·threshold 선택 ·champion 등록	model.joblib, metadata.json, metrics.json

재현성 검증: 현재 dvc status는 "Data and pipelines are up to date"를 반환해 커밋된 코드·데이터·산출물이 정합함을 보장한다. dvc.lock에는 각 stage 입출력의 해시가 고정되어 있어, 다른 시점·다른 PC에서도 동일 파이프라인 결과를 재현할 수 있다.

원격 저장소(remote): week03 강의의 local-folder 방식을 따라 sibling 폴더를 DVC default remote(localstorage)로 설정했다. dvc push로 캐시 객체 49개를 remote에 업로드하고, clean clone 상태에서 dvc pull로 데이터·모델이 그대로 복원되는 것을 검증했다(dvc status -c로 cache-remote

동기화 확인). 공유 클라우드 remote(S3/GCS 등)는 별도 인프라가 필요해 본 MVP 범위에서는 제외했고, 향후 과제로 둔다.

3.3 Experiment Tracking & Model Governance (MLflow)

추적 아키텍처: MLflow는 backend store(메타데이터)와 artifact store(파일)로 구성된다. 본 프로젝트는 추가 서버 없이 로컬에서 운영하기 위해 backend를 SQLite(sqlite:///mlflow.db)로, artifact는 로컬 파일시스템으로 두었다. 경량 SQLite 백엔드는 모델 단계 전이(stage transition)와 태깅을 지원하므로, 단일 PC 환경에서 실험 비교와 간단한 모델 거버넌스를 모두 충족한다.

Run 단위 기록: 모든 학습/튜닝은 하나의 MLflow run으로 기록된다. run마다 (1) 하이퍼파라미터(params), (2) validation/test의 F1·ROC-AUC-recall-precision 등 지표(metrics), (3) 아티팩트(직렬화 모델, confusion matrix·ROC·SHAP 플롯, normal/defect 예시 요청)를 함께 로깅한다. 덕분에 baseline·튜닝 run을 같은 화면에서 정량 비교하고, 결과를 코드 변경과 연결해 추적할 수 있다.

Champion 거버넌스(승격 라이프사이클): tune_logistic은 최종 모델 run에 champion 및 serving_candidate=true 태그를 부여한다. 새 champion이 등록되면 동일 experiment의 이전 champion run은 자동으로 superseded로 강등되어, 서빙 후보가 항상 정확히 하나만 유지된다. 이는 모델 레지스트리의 "Production 단계 단일화" 패턴을 경량으로 구현한 것으로, 서빙 측(FastAPI/Docker)은 champion 아티팩트만 신뢰하면 되도록 책임을 분리한다.

현재 champion run ID는 **c902f8c099e941e6ba2d3dc62a4cf3b1**이며, 모델·데이터 버전과 threshold(0.49)가 metadata.json에 함께 고정되어 서빙 시점까지 일관되게 전달된다.

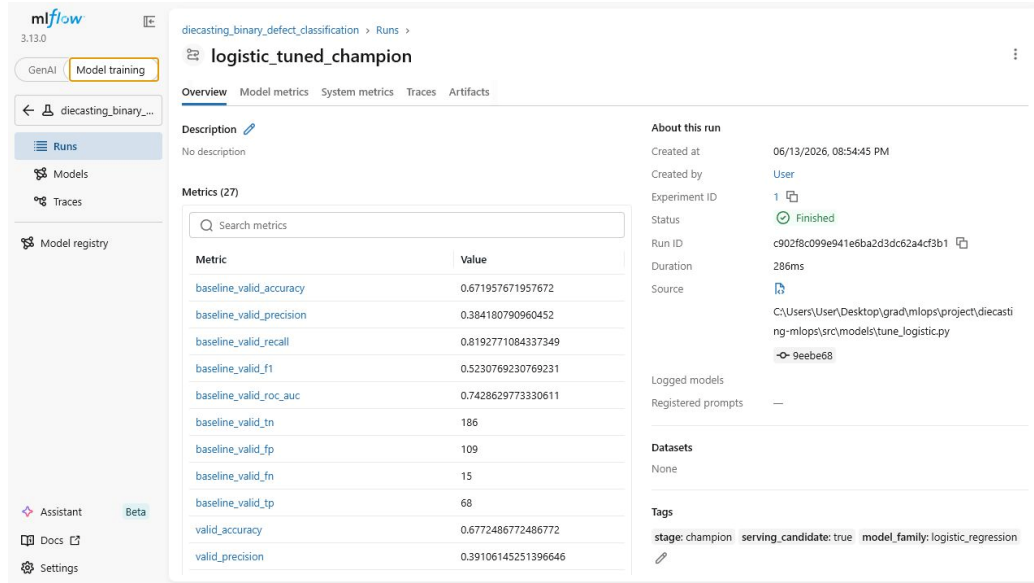


그림 3-1. MLflow champion run — 파라미터·지표·태그(run ID c902f8c0...)

3.4 컨테이너화 (Docker)

설계 의도: week15 자료의 핵심 요구는 production 클라우드 배포 자체가 아니라 실행 재현성과 안정적인 serving demo다. 이에 champion 추론 환경을 API 코드·Python runtime·고정 버전 의존성·champion 아티팩트로 묶어 단일 image로 고정했다. 로컬 .venv가 없어도, 다른 PC에서도 동일한 추론 결과를 보장하는 것이 목표다.

추론 전용 최소 image: 이미지는 의도적으로 추론에 필요한 것만 포함한다. 포함: src/api, model.joblib, metadata.json, feature_importance.json, 고정 API 의존성(requirements-api.txt). 제외: raw/processed 학습 데이터, DVC 캐시·MLflow DB, 학습·튜닝·EDA 코드, Streamlit UI. .dockerignore로 빌드 컨텍스트를 최소화해(약 202KB) image 크기를 약 154MB로 유지했고, HEALTHCHECK로 컨테이너 상태를 모니터링한다. 빌드·실행·API 검증의 상세 결과는 6장에서 다룬다.

3.5 End-to-End 재현성 보장

네 계층은 다음과 같이 맞물려 “코드 한 커밋 → 동일한 데이터 → 동일한 모델 → 동일한 서빙 환경”을 보장한다.

- **코드+설정:** Git 커밋 + configs/params.yaml 이 실행 로직과 파라미터를 고정한다.
- **데이터+파이프라인:** dvc.lock 해시가 입력 데이터와 각 stage 산출물을 고정하고, dvc pull 로 어디서나 복원한다.

- **실험+모델 선택:** MLflow run 이 지표·아티팩트를 기록하고 champion 태그가 서빙 대상을 단일화한다.
- **실행 환경:** Docker image 가 runtime·의존성·champion 아티팩트를 동결해 추론 결과를 환경 독립적으로 재현한다.

결과적으로 임의 시점의 모델을 “git checkout + dvc pull + docker run” 세 단계로 재현할 수 있으며, 이것이 단순 성능 비교를 넘어선 본 프로젝트의 MLOps 기여다.

4. 모델 실험 및 성능 비교

4.1 AutoML-lite Leaderboard

동일한 split 위에서 5개 후보 모델을 학습·평가하고 validation F1(동점 시 ROC-AUC)을 기준으로 순위화하는 AutoML-lite leaderboard를 구현했다. 모든 모델은 class_weight='balanced'로 불균형을 보정했다.

Model	Val F1	Val AUC	Val Recall	Test F1	Test AUC
Logistic Regression	0.523	0.743	0.819	0.494	0.758
Decision Tree	0.486	0.749	0.819	0.467	0.690
XGBoost	0.467	0.731	0.602	0.424	0.719
Random Forest	0.467	0.746	0.590	0.487	0.745
LightGBM	0.389	0.715	0.410	0.365	0.688

Champion 선정 근거: Logistic Regression이 validation F1 0.523으로 최고였다. 불량 탐지에서 중요한 recall(0.819)이 높고, 학습·추론이 가장 빠르며(<0.01s), 선형 모델이라 계수 기반 해석이 용이하다. 트리/부스팅 계열은 recall이 낮거나(RF/XGB ~0.59~0.60) F1이 떨어져, 본 데이터·목표에는 단순 선형 모델이 가장 적합했다.

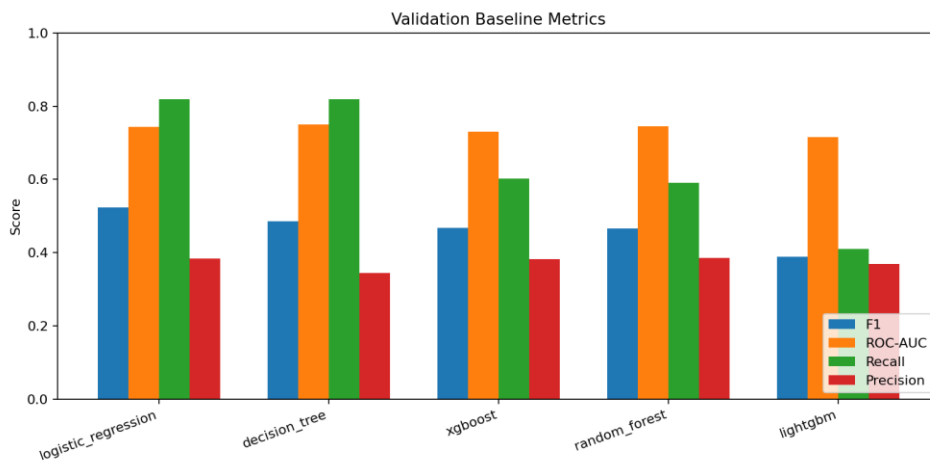


그림 4-1. 5개 baseline 모델 validation 지표 비교

4.2 Logistic Regression 튜닝

선정된 Logistic Regression을 StratifiedKFold(5-fold) CV로 튜닝했다. solver-penalty 조합과

C(logspace $-3 \sim 3$), class_weight를 탐색공간으로 두고 deterministic하게 30회 후보를 평가(F1 기준)했다. 최적 파라미터는 다음과 같다.

하이퍼파라미터	값
C (역정규화 강도)	0.0562
penalty	l2
solver	saga
class_weight	balanced

4.3 결정 임계값(Threshold) 튜닝

validation 확률에 대해 0.05~0.95 구간을 0.01 step으로 sweep하며 F1을 최대화하는 임계값을 선택했다(동점 시 recall→precision→0.5와의 거리). 최종 임계값은 0.49이다. 기본 0.50 대비 recall이 향상되어 불량 누락(false negative)을 줄였다.

구분	F1	ROC-AUC	Recall	Precision
Baseline validation @0.50	0.523	0.743	0.819	0.384
Tuned validation @0.49	0.534	0.748	0.843	0.391
Champion test @0.49	0.488	0.758	0.759	0.360

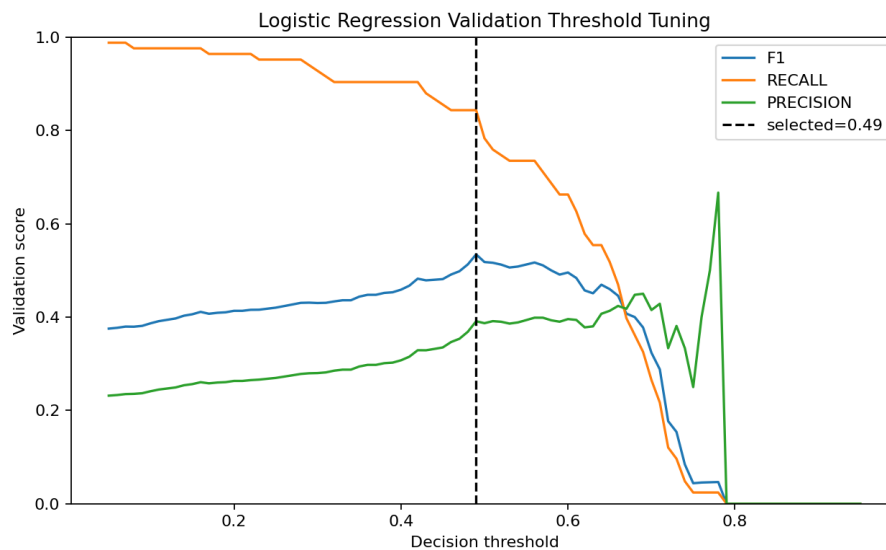


그림 4-2. 임계값 sweep에 따른 지표 변화 (선택값 0.49)

MLflow champion run ID: c902f8c099e941e6ba2d3dc62a4cf3b1. 최종 모델은 train split만으로 적

합되어 valid/test 누출 없이 평가되었다.

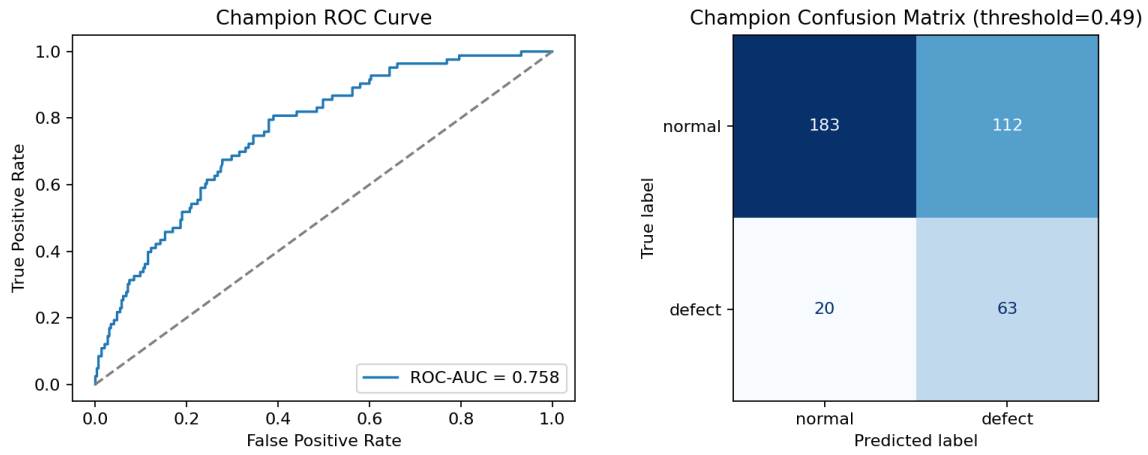


그림 4-3. Champion 모델 ROC curve(좌)와 confusion matrix(우, test set)

5. XAI 및 오류 분석

5.1 전역 해석 (Global Explanation)

SHAP(LinearExplainer)와 permutation importance를 함께 사용해 모델의 전역 판단 근거를 분석했다. 두 방법 모두 Factory_Humidity, Spray_1_Time, Spray_Time을 최상위 기여 변수로 지목해 해석의 일관성을 확인했다.

Feature	평균 SHAP	효과 방향
Factory_Humidity	0.779	값이 높을수록 → normal
Spray_1_Time	0.489	값이 높을수록 → defect
Factory_Temp	0.393	값이 높을수록 → normal
Spray_Time	0.358	값이 높을수록 → normal
Velocity_2	0.244	값이 높을수록 → normal
Air_Pressure	0.188	값이 높을수록 → defect
Coolant_Temp	0.187	값이 높을수록 → normal
Cylinder_Pressure	0.133	값이 높을수록 → normal

permutation importance(test set, F1 기준) 상위는 Factory_Humidity(0.079), Spray_1_Time(0.035), Spray_Time(0.020), Coolant_Temp(0.010)로, EDA의 SMD 분석 및 SHAP 결과와 부합한다. 즉 공장 습도와 분사 시간이 불량 판정의 핵심 동인이다.

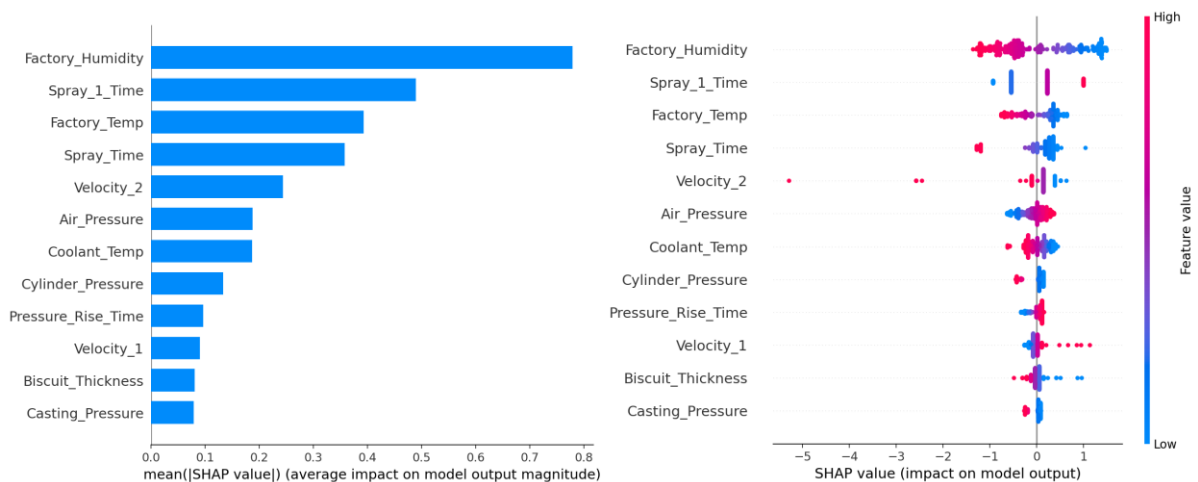


그림 5-1. SHAP 요약 — 평균 |SHAP| 막대(좌)와 beeswarm(우)

5.2 지역 해석 (Local Explanation)

개별 예측의 근거를 확인하기 위해, 모델이 가장 높은 확신으로 불량으로 판정한 test 샘플(index 57, 실제=불량, 예측=불량, defect 확률 0.801)을 SHAP waterfall로 분해했다.

Feature	입력값	SHAP	방향
Factory_Humidity	46.4	+1.391	불량 강하게 기여
Factory_Temp	36.1	-0.556	정상 쪽
Velocity_2	0.165	+0.512	불량 쪽
Spray_Time	8.0	+0.268	불량 쪽
Spray_1_Time	1.0	+0.228	불량 쪽

이 샘플은 낮은 공장 습도(46.4)가 불량 판정에 가장 크게 기여했으며, 이는 전역 해석(습도가 낮을 수록 불량)과 일치한다. 엔지니어는 이러한 변수별 기여도를 통해 '왜 불량으로 판정되었는지'를 확인하고 공정 조정의 단서를 얻을 수 있다.

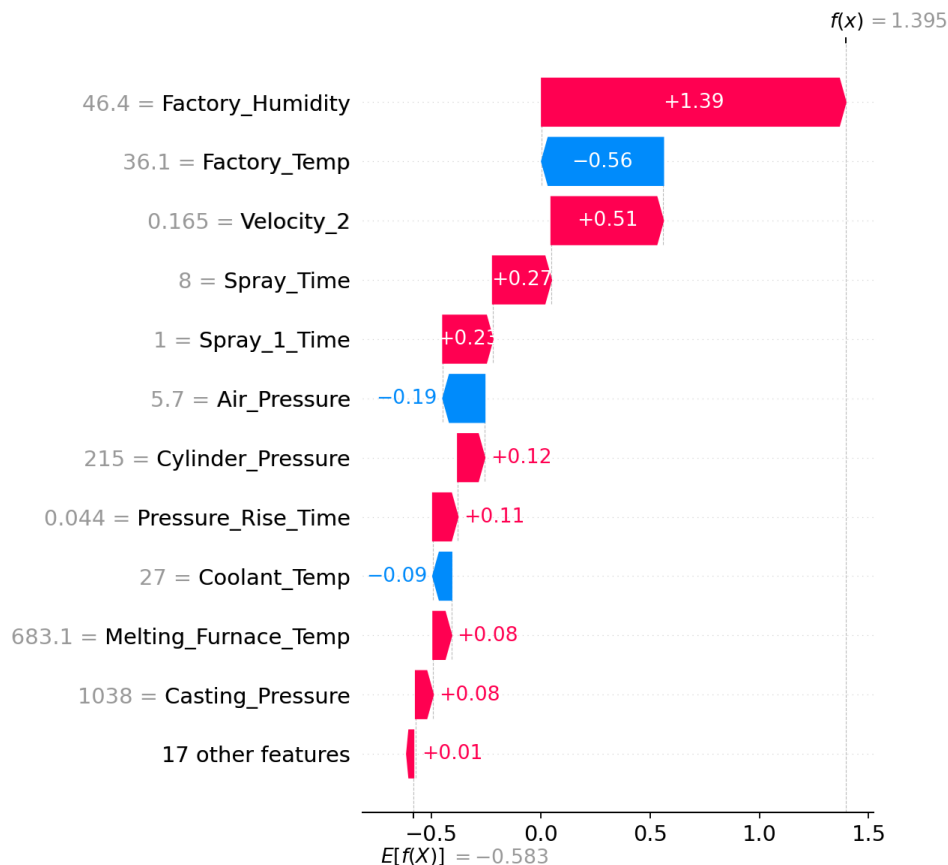


그림 5-2. 불량 샘플(index 57)의 SHAP waterfall 지역 해석

5.3 모델 한계와 개선 방향

- 낮은 **precision(test 0.36)**: recall 을 높이는 임계값(0.49)을 택해 false positive 가 많다. 불량 누락 비용이 큰 제조 맥락에서는 합리적이거나, 과검 비용이 크면 임계값 재조정이 필요하다.
- **class imbalance(22% 불량)**: class_weight 로 보정했으나 근본적 한계가 있어 SMOTE 등 리샘플링·비용민감 학습을 추가 검토할 수 있다.
- **다중공선성**: 압력 변수 간 상관이 매우 높아 계수 해석이 왜곡될 수 있다. feature 선택/차원 축소로 개선 가능하다.
- **선형 모델 한계**: 비선형 상호작용을 충분히 포착하지 못한다. 데이터가 늘면 부스팅 계열 재평가가 필요하다.

6. Serving 및 Demo

6.1 FastAPI Endpoint

champion 모델을 FastAPI REST API로 서빙한다. 세 개의 endpoint를 제공한다.

Endpoint	Method	기능
/health	GET	모델 로드 상태 확인 (status=ok)
/model-info	GET	모델·데이터 버전, threshold, feature 목록 반환
/predict	POST	feature 입력 → 예측·확률·상위 기여 feature 반환

/predict는 입력 feature를 DataFrame으로 변환해 모델 확률을 계산하고, metadata의 threshold(0.49)를 적용해 normal/defect를 결정한다. 응답에는 예측 라벨, 클래스별 확률, 상위 5개 기여 feature(입력값 포함), 모델·데이터 버전이 포함되어 추적성과 설명가능성을 함께 제공한다.

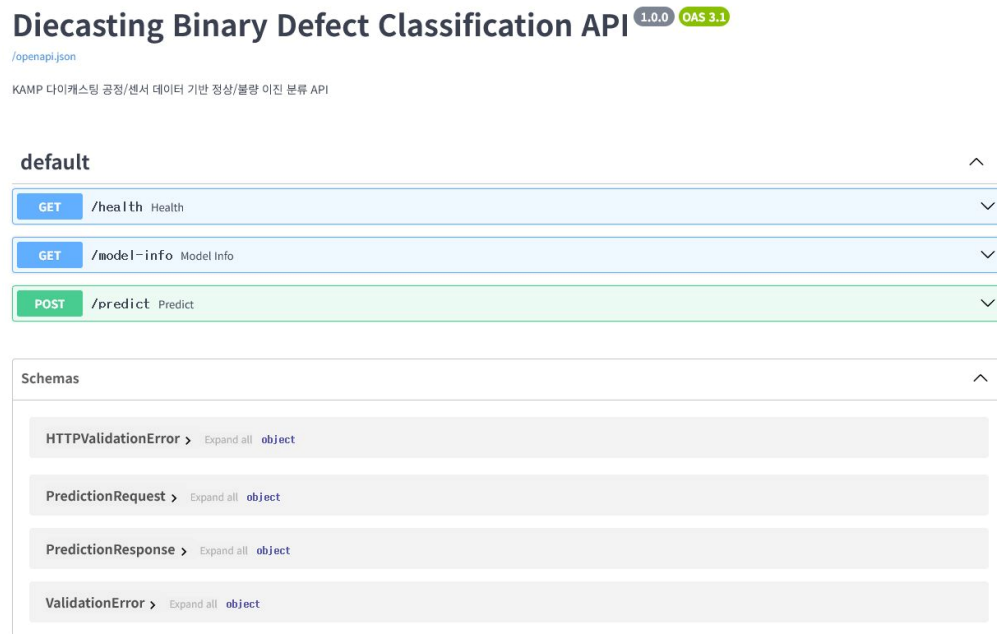


그림 6-1. FastAPI Swagger UI — /health, /model-info, /predict

6.2 Streamlit Web UI

비개발자도 사용할 수 있도록 Streamlit 대시보드를 6개 탭으로 구성했다.

- Predict: 고정 normal/defect 샘플로 /predict 호출, 예측·확률·상위 feature 표시

- Metrics: champion 버전·threshold·데이터 프로파일과 validation/test 지표, classification report
- Data EDA: 클래스 분포·상관 히트맵·분포/박스플롯·이상치 후보·상수 feature
- Feature Importance: 상위 N 개 permutation importance
- Baseline/XAI: 5 종 leaderboard 표·플롯과 SHAP 요약·지역 해석
- Artifacts: 산출물 파일 존재 여부·크기 점검

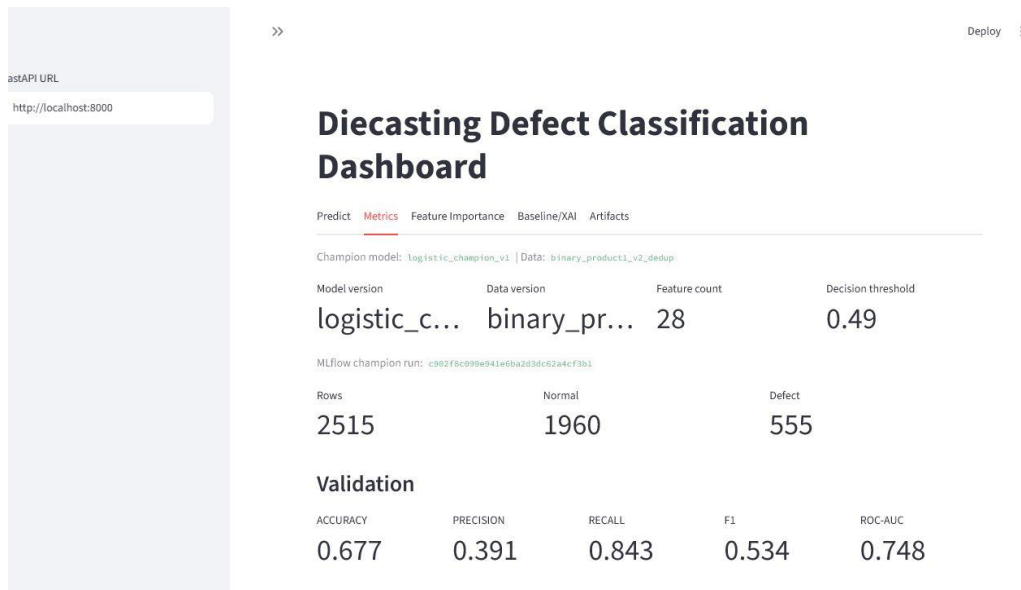


그림 6-2. Streamlit 대시보드 — champion 지표 화면

6.3 Docker 배포 및 검증

API 코드·Python runtime·고정 의존성·champion 아티팩트를 하나의 image로 고정해, 로컬 .venv와 독립적으로 동일한 추론 환경을 재현한다. 이미지는 추론 전용이며 학습/DVC/MLflow/Streamlit은 포함하지 않는다.

항목	값
Image	diecasting-api:logistic-champion-v1
Base runtime	python:3.10-slim
Image size	약 154 MB
Health status	healthy (volume 없이 image 자체 아티팩트로 실행)
/health	status = ok

항목	값
/model-info	logistic_champion_v1 / binary_product1_v2_dedup / threshold 0.49
normal /predict	normal, normal 확률 0.8937
defect /predict	defect, defect 확률 0.6982

DVC pull로 champion 아티팩트를 복원한 뒤 docker build → docker run으로 컨테이너를 기동하고, 네 개 요청(/health, /model-info, 정상/불량 /predict)이 모두 HTTP 200으로 정상 동작함을 확인했다.

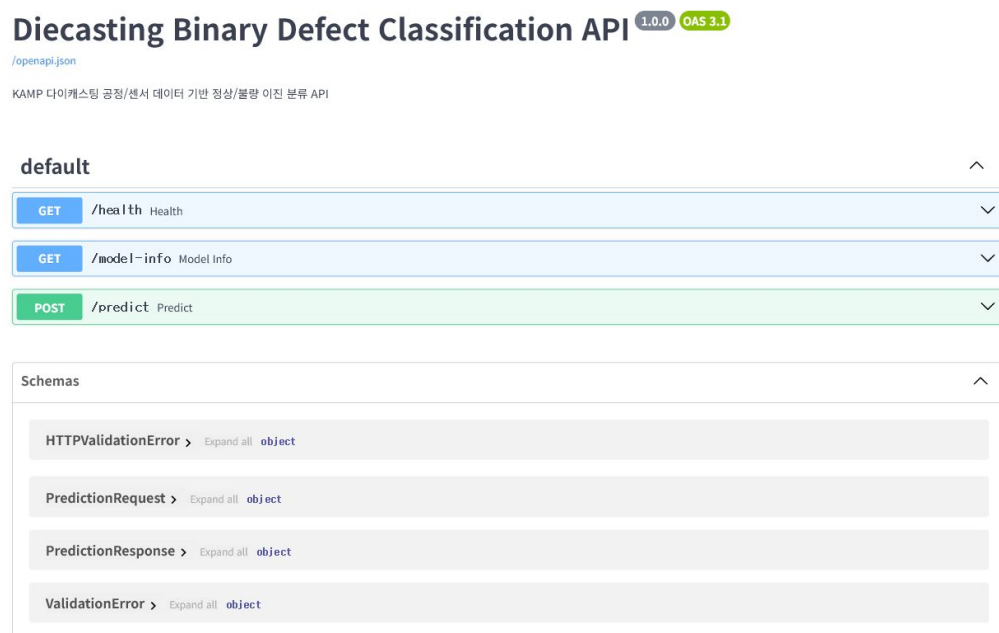


그림 6-3. Docker 컨테이너에서 제공되는 FastAPI Swagger UI

7. 결론 및 Future Work

7.1 결론

본 프로젝트는 다이캐스팅 불량 예측이라는 단일 문제를, 데이터 버전관리(DVC) → 실험 추적 (MLflow) → 모델 튜닝 → XAI → API/UI 서빙 → 컨테이너 배포로 이어지는 end-to-end MLOps 파이프라인으로 구현하고 각 단계를 실제로 검증했다. Champion 모델(Logistic Regression)은 test 기준 F1 0.488-recall 0.759-ROC-AUC 0.758로, 불량 탐지(recall)에 초점을 둔 운영 가능한 baseline을 제공한다. 무엇보다 dvc repro/pull과 Docker로 동일 결과를 재현할 수 있는 구조를 갖춘 것이 핵심 성과다.

9개 평가 요구 항목(문제정의·데이터 이해·소스 버전관리·데이터 버전관리·전처리/FE·AutoML/실험관리·성능비교·XAI·API/UI 배포)을 모두 충족했다.

7.2 Future Work

- 데이터/예측 분포 드리프트 모니터링과 알림 체계 도입
- 신규 데이터 누적 시 자동 재학습·재평가 트리거 및 champion 자동 갱신
- AutoGluon/H2O 등 본격 AutoML 도입으로 후보 모델 폭 확장
- 클라우드 registry push 및 원격 서버/오케스트레이션 배포, 모델 성능 모니터링
- 멀티클래스 결함 분류 및 비용민감/리샘플링 기반 불균형 대응 고도화